Programming Exercise – Vigenère Cryptography & Covering All Cases

Summary

Vigenère Cryptography - Applying security practices to classes reminds us to incorporate security into the classes we develop and program. There are so many security practices we can choose from. This exercise uses some of the material we have already been working on plus some cryptography. We are practicing confidentiality in this exercise.

Covering All Cases - Sometimes a user may not input a correctly formatted or withing policy input. Secure programming practices are to use defaults to handle cases not explicitly covered. This can violate other secure practices. For example, implementing a default password. Default passwords are common in the industry. You can search for the model of your router and default password to find out what the default password is. We are redefining default password and implementing a method to cover all cases using defaults to handle cases not explicitly covered.

Vigenère Cryptography

- Create a class to perform encryption and decryption.
    - Name the class cryptographer
    - Definition
        - Alphakey: the key used for characters of the alphabet
            - The Alphakey is ARGOSROCK
        - Numberkey: the key used for the numbers
            - The Numberkey for this exercise is 1963
- Create four methods for your cryptographer class
    - Method 1 – encrypt-vigenere
        - Read the input of a alphakey and the cleartext
        - Encrypt using the Vigenère Algorithm
    - Method 2 - decrypt-vigenere
        - Read the input of a alphakey and the ciphertext
        - Decrypt using the Vigenère Algorithm
    - Method 3 – encrypt-number
        - Read the input of a Numberkey and the cleartext

        - Encrypt using the Vigenère Algorithm
        - Refer to the Vigenere Number Table below
    - Method 4 - decrypt- number
        - Read the input of a Numberkey and the ciphertext
        - Decrypt using the Vigenère Algorithm
        - Refer to the Vigenere Number Table below
    - HINT - don't forget to account for lowercase and uppercase
    - HINT - if you have duplicate code in the two methods, make functions or methods inside this class and call them from the other methods.
    - HINT – ASCII is your friend.
- Program behaviors
    - Prohibit the use of non-alphabetic and non-numerical inputs.
        - In other words, the password should include only combinations of A-Z, a-z, or 0-9
        - You shall have at least 1 of each A-Z, a-z, and 0-9
    - Rewrite the code to store and read usernames and passwords in the array (database) with encrypted ciphertext from your cryptographer.
    - Minimum tests
        - Take the input usernames and passwords, encrypt them, compare them to the stored usernames and passwords which are encrypted.
        - Take the stored usernames and passwords, decrypt them, compare them to the cleartext input usernames and passwords.
    - Failures and passes should be handled in a secure software development way. In other words, do not leak information which the users or attackers could use to hack your program.

Reference: Numerical Vigenère Table

	0	1	2	3	4	5	6	7	8	9
0	0	1	2	3	4	5	6	7	8	9
1	9	0	1	2	3	4	5	6	7	8
2	8	9	0	1	2	3	4	5	6	7
3	7	8	9	0	1	2	3	4	5	6
4	6	7	8	9	0	1	2	3	4	5
5	5	6	7	8	9	0	1	2	3	4
6	4	5	6	7	8	9	0	1	2	3
7	3	4	5	6	7	8	9	0	1	2
8	2	3	4	5	6	7	8	9	0	1
9	1	2	3	4	5	6	7	8	9	0

Covering All Cases

- Create a class to perform default password.
  - Name the class default-password.
- Program behaviors
  - Change your software to allow the user two (2) attempts to create a valid password.
  - A valid password meets the password policy.
  - Create a method which returns the password policy.
  - Update your password validation code to user the method which returned the password policy.
  - If the user fails to create a valid password in two attempts, call the default-password class.
  - The default-password class
    - Creates a random password
    - The password must meet the password policy (see previous programming exercise)
    - The class notifies the user the password has been set to a default password, and the user will receive a secure email with the password.
  - Failures and passes should be handled in a secure software development way. In other words, do not leak information which the users or attackers could use to hack your program.

General Behaviors

- All the Programming 1 Exercise program behaviors.
- Programming language: Java or C++. Your choice
- Did you change the code from previous assignments?
    - That's ok if you did. Software evolves. What is important is configuration control.
    - You need to comment in the code to indicate changes made.
    - You need to document it in a configuration control document.
- Update your configuration control document (even if you did not make changes to past code)
    - Each time you change past code document the change and why. Include the requirements.
    - Include date, time, and version number of the software so we can track changes. This is important in secure software development!

Q&A

Is the coding project supposed to be run from the command line?

This is the choice of the developer.

Rules and Important Notes:

1. You must comply with AI use rules as specified in the syllabus.
2. If your code requires a special command to run, you must provide that in some instructions. Create a readme file with details.
3. Use only standard libraries. We will not install or set up any outside code or libraries.
4. You must comply with code reuse rules as specified in the syllabus.
5. Your work will be compared to other submissions, past and present.
6. Projects will be graded on whether they correctly solve the problem and whether they adhere to good programming practices.

You are required as a professional to effectively and efficiently present your work in a manner which is suitable for review and comprehension.

a. Code readability
    i. Fewest lines possible
    ii. Appropriate naming conventions
    iii. Code block segmentation
    iv. Proper formatting and indentation
    v. Proper Function cohesion and coupling
    vi. Don't repeat coding
    vii. Avoid deep nesting
b. Standardized Headers with appropriate details (name, date, creator, etc.). See example below.
c. Formalized exception handling

10. Projects must be submitted by the time specified on the due date. Projects submitted after that time will get a grade of zero.

Sample Header

```
/**
COP 4078 Exercise: 1
File Name: GuessingGame.java


The GuessingGame class provides the functionality of a number guessing game
where the program responds with a High/Low response and a Warmer/Colder
response for the guesses.

@author Stephen Hopkins
@version 1.0

*/
```